

Local AI Versus Cloud AI Is a Governance Decision

Set the limits before choosing where the model runs



Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Decide what each AI workflow must protect, may do, and needs to survive failure or exit before choosing where it runs.

Local-versus-cloud debates argue about models, the fastest-moving and least decisive variable in the stack. A better review runs seven boundary checks for each workflow: data, access, action authority, evaluation, resilience, support, and exit. Different workflows correctly land on different answers.

- 01** Local is not automatically private and cloud is not automatically resilient. Both properties come from operating habits, not the hosting choice.
- 02** Separate where a model runs from what it may do: write down what the system may recommend, prepare, approve, execute, or reverse before selecting any architecture.
- 03** Plan and price the exit while it is still cheap to design, and test it before signing, because an untested exit may fail when it is needed.

The argument

Local, cloud, hybrid, or no deployment at all. The answer should follow from a boundary decision about seven things: data, access, action authority, evaluation, resilience, support, and supplier-and-exit. Model fashion is not a governance test.

The deployment debates I get asked to referee usually arrive already broken. One camp has benchmarked the frontier cloud models and wants the best one. The other has a security instinct that anything sensitive belongs on hardware the company can touch. Both camps argue about models. Neither has answered the questions that actually decide the matter. Where may this workflow's data move? Who can reach the system and its admin controls? What is it permitted to do? How will its behavior be checked as everything underneath it changes? What happens when it fails? Who runs it in year three? And how does the company leave? I have watched architecture reviews spend three meetings on model quality and thirty minutes on all seven combined — an inversion the resulting incidents eventually correct.

The order should be reversed. Deployment pattern is a governance outcome, not a technology preference. The right answer falls out of the boundaries the workflow needs — and different workflows in the same company will land on different answers, correctly. NIST's generative-AI risk profile ties AI use to managed risk, including third-party components [1]; CISA's cloud security architecture requires adopters to identify which security responsibilities sit with them and which with the provider [2]. This paper turns those requirements into a working instrument — seven boundaries, examined per workflow, before any model is named. It is written for CIOs, architecture leaders, security leaders, AI program owners, and workflow executives. Most of the boundary categories are not novel; they reorient long-standing federal cloud guidance from procurement to per-workflow governance — NIST's definition of cloud computing and its

deployment models [3], its guidelines on security and privacy in public cloud computing [4], and its cloud computing synopsis and recommendations [5].

Why the model-first debate fails

Model-first reasoning fails because model properties are the fastest-moving thing in the stack. Rankings shuffle quarterly. Prices move monthly. The edge that justified a choice erodes while the purchase paperwork is still moving. Boundaries have the opposite lifespan. The data-residency rule, the customer commitment, the regulator's expectation, and the company's operating capacity all change slowly. Anchor a long-lived decision to the fastest-moving variable, and the decision is stale at delivery.

Model-first reasoning also smuggles in false safety equations. "Local means private" survives until the endpoint audit. An on-premises model with weak access control, unpatched serving software, and admin sprawl is less private than a well-governed cloud tenant. "Cloud means resilient" survives until the first regional incident or quota surprise. Neither property comes from the deployment pattern. Both come from the operating discipline around it — which is exactly what the boundary review examines and the model bake-off does not. This is threat modeling applied to a deployment choice: what matters is the boundary — where data, access, and action authority cross from trusted to untrusted hands, and who answers on each side — which a label like "local" or "cloud" does not settle. "Local means private" holds only until an endpoint audit finds the real boundary somewhere the label never named.

Getting this wrong is rarely a catastrophe. It is expensive rework. The workflow built on a pattern its data classification never permitted, discovered at audit. The local cluster sized for a pilot and abandoned when the real load arrived. The cloud contract renewed under pressure because nobody had priced leaving. Each is a boundary question answered late, at the highest possible cost.

The seven-boundary deployment decision

Reversing that order turns the decision from a model comparison into a per-workflow boundary review with a clear owner. The workflow's risk owner runs the checks and signs the resulting decision. The instrument checks seven boundaries for one specific workflow — not for the enterprise in the abstract. Each boundary yields evidence, and the pattern is chosen to fit the evidence. Run honestly, the review sometimes concludes cloud, sometimes local, often hybrid, and occasionally that no current option meets the need responsibly. That last verdict is also a governance answer, and a cheaper one than learning it in production.

EXHIBIT 1**The deployment boundary map****Data and identity**

Trace real data, telemetry, backups, access, and attribution.

Action authority

Separate where the model runs from what the workflow permits it to do.

Evaluation and change

Choose the pattern whose behavior and change model can be tested and governed.

Resilience and support

Price failure, recovery, patching, and the operating roster through year three.

Supplier and exit

Define notices, portability, deletion proof, and a tested exit before commitment.

Data boundary

The first boundary traces the workflow's actual data through the system's actual anatomy. Not "the model" in the abstract. Prompts, retrieved documents, embeddings, logs, telemetry, fine-tuning sets, admin access, support channels, backups. Each moves data somewhere, under some retention rule, visible to some group of people. Reviews that stop at "the model is hosted in-region" routinely miss that the support pathway or the telemetry pipeline crosses a line the workflow's data may not.

Retention and secondary use are the map's fine print, and the fine print governs. A provider may process in-region and still keep prompts for abuse monitoring on a different clock. An embedding store may outlive the documents it was built from. A fine-tune may carry training data into a model artifact that travels on its own rules. None of this is hidden. It lives in the terms and the architecture diagrams. It only gets read when a decision instrument demands the map.

The boundary resolves into a data-flow map per option. Input, processing, retrieval, telemetry, admin, backup, deletion — each marked with its classification, its contract terms, and its verified deletion behavior. The decision rule is not local-by-default. Choose a bounded or local pattern when required controls cannot be demonstrated in the cloud. Choose cloud when its managed controls clearly beat what the company can operate itself. Real enterprises exercise both directions of that rule.

Access and identity boundary

The second boundary asks who can reach the system — models, data stores, tools, admin planes, and support channels — and whether that population matches the workflow's accountability needs. Assumptions do the damage here. A local install presumed private while a shared service account with a

year-old password reaches it. A cloud service presumed controlled because corporate login exists, while tenant admins and vendor support sit outside the story the workflow owner was told.

The AI-specific wrinkle: new access surfaces arrive with the technology and get missed by standing reviews. The prompt history that becomes a sensitive-data store nobody classified. The retrieval index that quietly grants broad read access to whatever was ingested. The evaluation set holding production samples on a data scientist's laptop. An identity review scoped to "the model" misses all three. The boundary review scopes to the workflow's full anatomy, which is where these surfaces live.

Ordinary identity discipline, applied to this new surface, settles it. Mapped identities and privileges, including service accounts. Tenant and admin boundaries. Separation of duties. Provisioning and offboarding records. Monitoring on privileged access. The rule is strict because everything downstream leans on it: reject any option whose identity model cannot support the accountability the workflow requires. An architecture that cannot say who did what is not eligible, whatever its benchmarks.

Action-authority boundary

The third boundary separates where a model runs from what it may do — two questions deployment debates keep fusing. Hosting decides data movement and operational control. Authority decides the stakes. A local model wired into production systems with broad permissions is a higher-stakes arrangement than a cloud model that can only draft text for human review. The reflex ranks them the other way, because "local" sounds contained.

What this boundary demands is an action envelope, written down separately from hosting. What may the system recommend, prepare, approve, execute, or reverse? What tool scopes and transaction limits apply? What approval records exist for high-stakes paths? Is there a tested kill path? No architecture selection until the envelope is clear — because the envelope drives requirements the pattern must serve, and choosing hosting first means retrofitting those requirements into whatever was bought.

Evaluation and change boundary

The fourth boundary asks how the company will know the system still behaves after everything changes. And everything changes. Cloud providers update models on their own schedules, and a static approval quietly ages underneath the workflow. Local deployments invert the failure: the model the team froze for stability becomes stale, unsupported, and unpatched, drifting from the data it serves. Both patterns need governed change. They need it differently.

The evidence per option: who holds update authority. What regression tests run against the workflow's own cases. What release evidence exists. How behavior is watched between releases. What triggers re-approval. Whether rollback is real. The rule favors honesty about capacity. Prefer the option whose change model the company can actually govern. A team that cannot staff model-lifecycle management should not own one, however attractive the control fantasy. A team whose vendor will not disclose change notices should not build reliance it cannot re-verify.

Resilience and continuity boundary

The fifth boundary prices failure. Uptime targets, latency floors, capacity peaks, incident response, and disaster recovery differ sharply by pattern. The recurring asymmetry: local capacity gets under-built because its costs are visible capital, while cloud dependence gets accepted without a workable fallback because its risks are contractual and abstract. Both errors surface in the same place — the workflow's worst week.

Latency belongs here too, because it is an operating requirement that gets mistaken for a performance preference. A workflow that needs sub-second response in a customer interaction has different eligible patterns than one that batches overnight. Quota ceilings and throttling under provider load are continuity facts a contract may only whisper. The review asks the workflow for its real tolerances and tests the options against them, instead of discovering the mismatch through user complaints.

The evidence is exercised, not asserted. Load tests at realistic peaks. Continuity drills with results. Dependency maps including the quiet ones: the identity provider, the network egress, the one engineer who understands the serving stack. Recovery objectives the operation actually accepts. And a defined degraded mode the workflow can genuinely run in. The rule allows layered answers: hybrid designs, bounded functionality during outages, or non-adoption where neither option can meet the need responsibly.

Support and operations boundary

The sixth boundary asks who runs the thing on an ordinary Tuesday in year three. Deployment patterns are also staffing decisions. A local stack needs serving expertise, patch discipline, hardware lifecycle, and on-call depth that most enterprises underestimate badly. A cloud arrangement needs vendor management, quota governance, cost visibility, and the skill to translate provider incidents into workflow decisions. Neither is free. They are differently shaped bills.

The evidence is a support model with names and numbers. Rosters and escalation depth. Skill coverage and its key-person risk. Expected ticket volume. Patch and upgrade cadence. And the budget line that funds it all past the founding team's enthusiasm. The rule is the same honesty test as change governance. Choose the pattern whose operating bill the company will actually pay — not the one whose bill it prefers not to look at.

Supplier and exit boundary

The seventh boundary examines the dependencies the choice creates — on models, hardware vendors, platforms, contracts, formats, and skills — and how the arrangement ends. Every deployment creates lock-in of some shape. Cloud lock-in is contractual and data-gravity. Local lock-in is capital, skills, and the sunk cluster nobody wants to strand. The governed question is whether the exit was designed and priced while designing it was cheap.

The evidence: exit clauses and what they were tested to mean. Data export actually run at real volumes. Replacement lead times. The asset and knowledge list a successor would need. And a named transition

owner. The rule treats an untested exit as a big commitment the company signed without reading. Concentration deserves explicit statement too: a single model family, a single region, and a single irreplaceable engineer are one risk with three addresses.

Two workflows, two verdicts

A pair of composite examples shows why the review must run per workflow. First: a claims-support tool for a regulated insurer. The data boundary binds hard. Case files carry health information under residency rules, and the vendor's telemetry pipeline fails the map. The action envelope is modest — recommend and prepare only — but identity rules are strict and auditors will want records they can check. The verdict lands on a bounded pattern: a dedicated tenant with telemetry controls in the contract for the general reasoning, a small local model for the extraction step that touches raw files, and an exit tested against a successor vendor's import format.

Second: a marketing-content assistant in the same enterprise. The data is low-sensitivity, the action envelope stops at drafting, and the volume is bursty. Here the resilience and support boundaries dominate. The team has no serving expertise and no appetite for patch discipline, while the cloud provider's managed controls exceed anything the function would build. Verdict: cloud, standard tenant, quarterly evaluation samples, exit priced at nearly zero because the workflow holds no state worth migrating.

Same company, same year, opposite answers — both correct. An enterprise-wide "AI deployment standard" that forced either workflow into the other's pattern would have manufactured either an audit finding or a wasted cluster. The unit of decision is the workflow, because the boundaries bind at the workflow.

How the seven boundaries interact

The boundaries are constraints to satisfy at once, not a scorecard to average, and their interactions decide the hard cases. Data and access constraints may force a bounded pattern that resilience economics argue against. That tension is the actual decision, and it belongs to the workflow's risk owner, made visibly. Action authority multiplies every other boundary's stakes: widen the envelope, and the evaluation, resilience, and identity requirements widen with it. Exit discipline keeps the whole decision revisitable — which matters because the model layer changes fastest.

Hybrid deserves precision, because it is the answer most reviews reach and the word covers three different designs. Split-by-step: sensitive steps run bounded or local, general steps run cloud — the claims example. Split-by-tier: a frontier cloud model for complex cases, a smaller bounded model for routine volume, with routing rules that are themselves governed. Split-by-mode: cloud in normal operation, a degraded local capability for continuity. Each carries its own boundary profile, and "hybrid" as a strategy statement, without naming which design, is a decision still unmade.

Run per workflow, the review also produces a portfolio pattern worth reading. The workflows clustering toward bounded local patterns show where the company's data constraints genuinely bind. The ones comfortable in cloud show where managed controls dominate. And repeated "no good option" verdicts

point at capability investments — often identity or evaluation capacity — that would open whole classes of adoption at once.

Shared responsibility deserves its own sentence in every cloud-leaning decision, because the phrase hides a moving line. Where the line falls depends on the service model and the provider; the customer must establish which controls it configures, which identities it governs, and which data it classifies, and it owns the workflow's behavior regardless [2]. Misconfigured settings and over-privileged accounts are the customer's to find and fix, whatever name the incident is reported under [2]. The boundary review makes the line explicit per workflow: this column the provider holds, this column we hold, and this column — usually evaluation and action authority — nobody holds until we assign it.

Cost honesty rounds out the comparison. Cloud costs are visible, metered, and politically noisy. Local costs are lumpy, capitalized, and hidden in salaries and depreciation. That biases casual comparisons toward whichever bill the company prefers not to see. The boundary review prices both patterns fully — support roster, evaluation cadence, patch discipline, exit — so the finance conversation happens on total operating cost, not on the shape of the invoice.

Running the review on two workflows first

The review installs modestly:

- Begin with two real workflows. Abstract enterprise-wide scoring hides the differences that matter.
- Complete the data and action maps before comparing models.
- Estimate the standing support, evaluation, security, and recovery load each option carries.
- Record the decision, the risk that remains, the triggers that reopen it, and the exit duties.

The evaluation boundary is also where whole classes of stalled adoption get released. Companies discover mid-review that the real constraint is having no way to test model behavior against their own cases — not data or hosting, and no deployment pattern fixes it. Build that capacity once, as shared infrastructure, and a dozen stalled deployment debates become decidable questions. The review surfaces this because it asks the evaluation question per workflow and gets the same embarrassed answer twelve times.

Two workflows, deliberately different — one data-sensitive and narrow, one broad and low-stakes — teach the instrument faster than any template rollout. The first pass on each takes days, not months. Most of the evidence already exists in pieces: the classification policy, the identity design, the vendor contract. The review's work is assembling fragments into a decision, and its first visible product should be a one-page decision record the workflow's owner signs.

The record's reopen triggers keep the decision alive without keeping it open. A new model class that changes the capability equation. A policy or regulator shift. A provider change notice above a threshold. An incident. A support-capacity change. Any trigger reopens the affected boundary — not the whole debate. That is how an organization tracks a fast-moving field without re-litigating its architecture quarterly.

Purchasing is where the boundary review earns hard cash, because contracts are boundary instruments. A team that arrives at the vendor table with its data map, action envelope, and exit requirements negotiates specific clauses: telemetry handling, change-notice windows, export formats, deletion verification. A team

that arrives with a model preference negotiates price. The first team's contract is a governance asset for years. The second team's contract is a discount on terms it never read closely.

Regulated industries will recognize the structure here, and that recognition is the point. The seven boundaries are the AI-shaped version of questions their control frameworks already ask about any material system. Data handling. Access. Authorization. Change management. Continuity. Operations. Third-party risk. Framing AI deployment this way lets the review ride existing governance machinery instead of building a parallel bureaucracy — and it gives risk and audit functions a vocabulary they already trust.

What the record should show

The practice leaves a record anyone can check:

- Decision records per workflow, signed by the risk owner, with residual risks stated.
- Data-flow and action-envelope maps that match the deployed reality.
- Change events handled by the governed path — regression run, reapproval where triggered.
- Continuity and degraded-mode drills run on schedule.
- Exit tests actually run: exports tried, hand-over lists current.
- Reopen triggers that fired, versus decisions quietly aging.

The last line is the health check for the whole practice. Deployment decisions in this field have a shelf life. A portfolio whose decision records show no recent trigger activity is not stable. It is unexamined — and the difference becomes visible at the least convenient moment available.

The strongest counterargument

Local is not automatically private. Cloud is not automatically resilient. Hybrid is not automatically balanced. Outcomes depend on how well the pattern is built and run. The framework supports a reasoned choice. It does not crown one pattern for everyone.

EXHIBIT 2

Hybrid is three designs, not one word**Split by step**

Sensitive steps run bounded or local; general steps run cloud.

Split by tier

A frontier model for complex cases; a smaller bounded model for volume.

Split by mode

Cloud in normal operation; a degraded local capability for continuity.

The rule

Name which design. 'Hybrid' alone is a decision still unmade.

A second fair objection: small companies cannot staff seven-boundary reviews. True — and the instrument scales down honestly. A twenty-person company runs the same questions in an afternoon, answers most of them "cloud, standard terms, minimal envelope," and writes half a page. The value at that scale is not the analysis but the two questions small companies otherwise skip entirely: what the system may do, and how they would leave.

The objection that seven boundaries is heavyweight for a fast-moving field has the sequence backward. The boundaries are what make speed safe. A workflow with mapped data flows, a defined action envelope, and a tested exit can change models in a week, because the change touches the volatile layer while the governed layers hold. It is the company without boundaries that cannot move. Every model change reopens every question, and architecture debate becomes the standing tax on adoption.

The governance decision underneath

The shift is from "which model should we run, and where?" to "which boundaries must hold, and which pattern holds them?" Ask it in that order and the deployment question stops being a technology argument. It becomes what it always was underneath: a governance decision about data, authority, failure, operations, and commitment. Decided per workflow. Decided on evidence. With the exit priced before the entrance is signed.

Model fashion will keep moving. That is the one certainty in the stack. Companies that govern the boundaries get to treat that movement as opportunity: swap the volatile layer, keep the governed ones. The ones that argue about models get to treat every movement as a recurring crisis, because nothing beneath the argument was ever settled.

The instrument's final virtue is that it produces decisions someone signed. In a field this noisy, the difference between an architecture and an accident is a record. Which boundaries were examined. What the evidence showed. Who accepted the remaining risk. What would reopen the question. That record turns deployment from a debate the company keeps having into a decision it keeps improving.

Notes and sources

- [1] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, July 26, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [2] Cybersecurity and Infrastructure Security Agency, United States Digital Service, and Federal Risk and Authorization Management Program, "Cloud Security Technical Reference Architecture," Version 2.0, June 2022. https://www.cisa.gov/sites/default/files/2023-02/cloud_security_technical_reference_architecture_2.pdf
- [3] Peter Mell and Timothy Grance, "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, September 2011. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>
- [4] Wayne Jansen and Timothy Grance, "Guidelines on Security and Privacy in Public Cloud Computing," NIST Special Publication 800-144, December 2011. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-144.pdf>
- [5] Lee Badger, Tim Grance, Robert Patt-Corner, and Jeff Voas, "Cloud Computing Synopsis and Recommendations," NIST Special Publication 800-146, May 2012. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-146.pdf>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.